CONFIDENTIAL — COMPANION TO THE FULL AUDIT REPORT

SUBSIDY DROPOFF

Numbered Issues, *Verbatim Fixes* & the 30-Day Plan

Every material issue found in the full audit, numbered and grouped by category, each with a ready-to-implement fix — exact code, exact config, exact header values. Followed by the complete competitive landscape, a compliance-aware monetization map, and a day-by-day 30-day remediation and content plan.

DOMAIN	AUDIT DATE	ISSUES NUMBERED	QUICK WINS / LONGER-TERM
subsidydropoff.com	August 31, 2026	46	39 / 7

CONTENTS

Document Map

HOW TO USE THIS DOCUMENT

Severity & Priority Legend

Severity reflects impact if left unfixed. The quick-win / longer-term tag reflects effort, not importance — several quick wins below (Issues 1–2) are the single highest-leverage fixes in this report.

CRITICAL breaks the product or its distribution outright

HIGH materially undermines trust, function, or discoverability

MEDIUM a real gap against a Fortune-500 bar

LOW hardening / polish

QUICK WIN hours, not days — a config value, a small function, a header line

LONGER-TERM requires design assets, content authoring, or sustained effort

SEVERITY	COUNT
Critical	3
High	11
Medium	19
Low	13

ISSUES 1–46

Numbered Issues & Verbatim Fixes

Category A — Operational Emergency — Deploy & Data Pipeline

> Fix this week. Everything else in this report is downstream of these seven items.

01 Deploy pipeline broken for 22 consecutive days — Cloudflare API token authentication failure, zero self-heal

CRITICAL QUICK WIN

The only successful deploy in the repository's 10-run CI history is 2026-08-09 (commit 8f4750b). Every scheduled run since (08-10, 08-17, 08-24, 08-31) has failed identically at the wrangler deploy step with “Authentication error [code: 10000]” — the CLOUDFLARE_API_TOKEN repository secret has been invalid for three weeks, and the weekly cron built to self-heal this has fired four times and failed four times.

VERBATIM FIX

```
1) Cloudflare dashboard → My Profile → API Tokens → Create Token → Custom token, scoped to exactly “Account → Cloudflare Pages: Edit” for the account owning subsidydropoff.com, with an explicit expiry date (not “no expiration”).
2) Update the secret:
   gh secret set CLOUDFLARE_API_TOKEN --repo <owner>/health-market
   (and confirm CLOUDFLARE_ACCOUNT_ID is still correct the same way)
3) Re-trigger immediately rather than waiting for next Monday:
   gh workflow run deploy.yml --repo <owner>/health-market -f planYear=2026
   gh run watch --repo <owner>/health-market
4) Record the token's expiry date in DEPLOY.md's “One-time setup” section and calendar a renewal reminder before it — silent token expiry is what caused this exact 3-week outage.
```

02 SLCSP crosswalk 404s at CMS's Socrata portal — a single failed source aborts ETL entirely

CRITICAL QUICK WIN

src/etl/sources.ts resolves the SLCSP county-ZIP crosswalk to https://data.healthcare.gov/resource/yaaf-rjhy.csv, which now returns a genuine HTTP 404 (the dataset ID has likely been renamed or retired). Because src/etl/run.ts loads all four required sources in one Promise.all, this single 404 aborts the other three datasets too — zero shard files are written, not a degraded set. The moment the token in Issue 1 is fixed, the next “green” deploy will ship zero benchmark data and every /api/estimate call will 503.

VERBATIM FIX

```
1) Find the current dataset ID: browse https://data.healthcare.gov/dataset/ for the ZIP-to-county crosswalk
(its API tab shows the live 4x4 ID after any republish), then update src/etl/sources.ts:

  slcspCountyZip: {
    ...same fields...,
    landingPage: "https://data.healthcare.gov/dataset/<NEW-SLUG>",
    downloadUrl: socrata("data.healthcare.gov", "<NEW-4X4-ID>"),
    urlVerified: true,
  }

2) Make each source's failure individually attributable in src/etl/run.ts — replace the bare Promise.all with
Promise.allSettled:

  const keys = ["planAttributesPuf", "ratePuf", "qhpLandscapeIndividualMedical", "slcspCountyZip"] as const;
  const results = await Promise.allSettled(keys.map((k) => load(k, args)));
  const failed = results.flatMap((r, i) => (r.status === "rejected" ? [keys[i]] : []));
  if (failed.length > 0) {
    for (const k of failed) console.error(`  FAILED to load ${k}`);
    throw new Error(`ETL aborted: failed to load ${failed.join(", ")}.`);
  }
  const [attributesCsv, ratesCsv, landscapeCsv, zipCsv] = results.map(
    (r) => (r as PromiseFulfilledResult<string>).value,
  );

3) Wire the existing zero-shard warning into an alert (see Issue 3) instead of failing silently: in deploy.yml
add `id: data-status` to the “Report data status” step and set `shards_empty=true`/`false` via $GITHUB_OUTPUT
in its else/if branches.
```

03 No failure alerting anywhere in the deploy workflow — a 22-day outage produced zero actionable signal

HIGH QUICK WIN

deploy.yml has no notify/Slack/webhook/issue-creation step and no `if: failure()` step anywhere. This is the structural reason the token-expiry outage in Issue 1 ran silently for 22 days across four scheduled failures, and it is why a future zero-shard deploy (Issue 2) would also go unnoticed.

VERBATIM FIX

Add `issues: write` to the top-level `permissions:` block (currently `contents: read` only), then add this step to the deploy job after “Deploy to Cloudflare Pages”:

```
  - name: Alert on deploy failure or empty dataset
    if: failure() || steps.data-status.outputs.shards_empty == 'true'
    run: |
      gh issue create \
        --repo ${{ github.repository }} \
        --title "Deploy issue: run ${{ github.run_id }} (${{ github.event_name }} on ${{ github.ref_name }})" \
        --body "Run failed or produced zero benchmark shards at $(date -u +%FT%TZ). See https://github.com/${{ github.repository }}/actions/runs/${{ github.run_id }}." \
        --label ops \
        || echo "issue creation failed, check gh auth"
    env:
      GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

No new secret needed — GITHUB_TOKEN is already available once issues:write is granted. For a paging channel instead, swap the gh issue create body for a slackapi/slack-github-action@v1.27.0 call against a SLACK_DEPLOY_ALERTS_WEBHOOK secret.

04 /api/health reports {ok:true} unconditionally, giving false assurance through the exact outage happening right now

MEDIUM QUICK WIN

src/api/handler.ts implements /api/health as a static {ok:true, supportedPlanYears, provider} with no dependency on when the underlying data was last successfully built. DEPLOY.md tells an operator to treat {"ok":true} as proof the deploy worked — but that field reads true throughout the entire 22-day token outage and would read true again the moment a zero-shard deploy ships, because the endpoint only proves the Worker is running, not that the data behind it is current.

VERBATIM FIX

```
1) src/core/benchmark.ts — extend the provider interface: `getFreshness?(planYear: PlanYear): Promise<{
generated: string } | null>;`
2) src/data/file-loader.ts — add a method reading the already-written index.json:
   `async loadIndex(planYear) { const r = await this.fetcher.fetch(new
Request(`https://assets.local${this.basePath}/${planYear}/index.json`)); return r.ok ? await r.json() : null;
}`
3) src/data/shard.ts — on StaticBenchmarkProvider: `async getFreshness(planYear) { const idx = await
this.loader.loadIndex?.(planYear); return idx ? { generated: idx.generated } : null; }`
4) src/api/handler.ts — extend the /api/health branch:

  if (url.pathname === "/api/health") {
    const freshness = await provider.getFreshness?.(SUPPORTED_PLAN_YEARS[0]);
    const ageDays = freshness ? (Date.now() - new Date(freshness.generated).getTime()) / 86_400_000 : null;
    return json({
      ok: true,
      supportedPlanYears: SUPPORTED_PLAN_YEARS,
      provider: provider.name,
      dataGenerated: freshness?.generated ?? null,
      dataAgeDays: ageDays === null ? null : Math.floor(ageDays),
      dataStale: ageDays === null ? true : ageDays > 21,
    }, 200);
  }

Point an external uptime monitor at `dataStale === false`, not just HTTP 200, so staleness itself pages
someone.
```

05 No staleness gate in the API response path — a frozen dataset is served indefinitely with full confidence

MEDIUM QUICK WIN

Every benchmark quote carries sourcePublished, but nothing in src/api/handler.ts compares it against the current date. UnverifiedDataError already models exactly this class of problem correctly for “dataset never loaded” (503, not 500) — there is no equivalent for “dataset loaded once, then the weekly refresh that keeps it current broke,” so a three-week-old snapshot is served today as an ordinary HTTP 200.

VERBATIM FIX

```
In src/api/handler.ts's estimate(), immediately after the `if (!isQuote(benchmark))` early return (benchmark is
now a confirmed BenchmarkQuote with sourcePublished available):

  const STALE_AFTER_DAYS = 21; // one weekly cron cycle (7d) + 2 missed-run grace
  const ageDays = (Date.now() - new Date(benchmark.sourcePublished).getTime()) / 86_400_000;
  if (ageDays > STALE_AFTER_DAYS) {
    throw new UnverifiedDataError(
      `Benchmark data for this region was last built ${benchmark.sourcePublished} ` +
      `(${Math.floor(ageDays)} days ago), exceeding the ${STALE_AFTER_DAYS}-day staleness threshold. ` +
      `The weekly refresh pipeline may be broken.`,
    );
  }

UnverifiedDataError is already imported and already maps to HTTP 503, so this reuses the existing “service
fine, dataset incomplete” contract.
```

06 public/data index.json overstates shard coverage — its counts are taken before validation, not from what was written

MEDIUM QUICK WIN

In src/etl/run.ts, index.json's shardCount and prefixes are computed from the pre-validation shard map, not from what the write loop actually persisted after validateShard() rejections. Today this has zero live impact (the API loads shards directly and re-validates on read), but any future ops dashboard or health check built against index.json would report phantom coverage, and the validation-failure count never leaves the console log.

VERBATIM FIX

```
In src/etl/run.ts, build index.json from what was actually written, and surface the dropped-shard count:

  let failures = 0;
  const writtenPrefixes: string[] = [];
  for (const [prefix, shard] of outcome.shards) {
    const problems = validateShard(shard);
    if (problems.length > 0) {
      failures += 1;
      console.error(`  FAILED ${prefix}: ${problems.slice(0, 3).join("; ")}`);
      continue;
    }
    writtenPrefixes.push(prefix);
    await writeFile(join(dir, `${prefix}.json`), JSON.stringify(shard));
  }

  await writeFile(join(dir, "index.json"), JSON.stringify({
    planYear: args.planYear,
    generated: new Date().toISOString(),
    shardCount: writtenPrefixes.length,       // was outcome.shards.size
    shardsFailedValidation: failures,          // new
    countiesBuilt: outcome.countiesBuilt,
    countiesSkipped: outcome.countiesSkipped,
    warnings: outcome.warnings,
    prefixes: writtenPrefixes.sort(),          // was outcome.shards.keys()
  }, null, 2));
```

07 No application-level rate limiting on /api/estimate

LOW QUICK WIN

Strict input validation exists, but there is no per-IP throttling, CAPTCHA, or request budget on a public, unauthenticated, computation-only endpoint. Cloudflare's platform DDoS protection still applies at the edge, so this is a minor gap — but a Fortune-500 property ships bespoke abuse control on a public JSON API by default.

VERBATIM FIX

```
Add a Cloudflare Rate Limiting Rule with no code change: dashboard → subsidy-dropoff Pages project → Security → WAF → Rate limiting rules, scoped to `(http.request.uri.path eq "/api/estimate")`, threshold 30 requests / 60s per IP, action Block for 60s.

Alternatively, bind an in-Worker limiter via wrangler.toml:

  [[unsafe.bindings]]
  name = "ESTIMATE_LIMITER"
  type = "ratelimit"
  namespace_id = "1001"
  simple = { limit = 30, period = 60 }

and in functions/api/[[route]].ts:
  const { success } = await env.ESTIMATE_LIMITER.limit({ key: request.headers.get("cf-connecting-ip") ?? "anon" });
  if (!success) return json({ ok: false, error: "rate_limited" }, 429);
```

Category B — Architecture Governance — Two Front-Ends, One CI Blind Spot

The parallel Next.js stack and its knock-on effects.

08 Two competing front-end stacks — only one deployed, with no migration governance or decision record

HIGH LONGER-TERM

wrangler.toml and the CI deploy step ship public/index.html + app.js + styles.css only. A parallel Next.js 16/Tailwind v4/shadcn rebuild (app/, components/) is wired into nothing — no output:"export", no Cloudflare adapter, no build step in CI — and has had no commits in 34 days. The migration intent (“the v0 build replaces this visual layer”) lives only in a commit message and a source comment, never in README.md or DEPLOY.md, so it has no owner and no target date.

VERBATIM FIX

```
Add a dated decision record to README.md:

  ## Architecture status (2026-08-31)
  Production = public/index.html + public/app.js + public/styles.css, deployed
  via `wrangler pages deploy public` (wrangler.toml). The app/, components/,
  lib/ tree (Next.js 16 + Tailwind v4 + shadcn) is NOT deployed and has had no
  commits since 2026-07-28 (dceaec8). Do not add product features there until
  either (a) a Cloudflare adapter is wired — output:"export" in next.config.mjs
  plus a Cloudflare Workers/Pages build step, and a cutover PR replaces public/
  wholesale — or (b) the tree is deleted:
  git rm -r app components lib public/photos design/v0-prompt.md next.config.mjs postcss.config.mjs
  and the next/react/react-dom/tailwindcss/recharts/motion/lucide-react/etc.
  dependencies are dropped from package.json.

Check Cloudflare's current guidance before wiring an adapter — they have moved this recommendation before (as of writing they favor the “vinext” Vite plugin for Next.js-on-Workers over the older @cloudflare/next-on-pages path).
```

09 CI's only type-check gate excludes the live production serverless handler entirely

HIGH QUICK WIN

tsconfig.json's include list covers the entire undeployed Next.js tree (app/, components/, lib/, src/) while its exclude list explicitly names “functions” — the directory Cloudflare Pages Functions loads from. deploy.yml's Verify job runs `npm run typecheck` against this config as the sole type-safety gate before every deploy, so it currently checks ~1,500+ lines that ship to nobody and is blind to the one file (functions/api/[[route]].ts) that runs on every real request.

VERBATIM FIX

```
Extend the stricter config that already exists (tsconfig.engine.json) rather than adding a new one:

  // tsconfig.engine.json
  "include": ["src/**/*.ts", "functions/**/*.ts"],

Then add a step to the Verify job in .github/workflows/deploy.yml, right after the existing Typecheck step:

  - name: Typecheck production handler (engine + functions)
    run: npm run engine:typecheck

engine:typecheck already exists as a package.json script but is currently never run in CI — this wires it in
without touching the Next tree's own tsconfig.json or typecheck script.
```

10 The site's own “friendly alias” redirects point at a route that does not exist in production, and there is no custom 404

HIGH QUICK WIN

public/_redirects defines `/calculator /plan 301` and `/cliff /plan 301`. /plan exists only as an unshipped route in the undeployed Next.js tree — public/ contains no file that would serve /plan, and there is no public/404.html either, so both aliases 301 straight into Cloudflare's generic unbranded 404 page. This is the only multi-URL navigation the site attempts, and it is broken today by the site's own configuration.

VERBATIM FIX

```
In public/_redirects, point the aliases at the page that actually exists until /plan ships:

  # Friendly aliases for the tool.
  /calculator / 301
  /cliff / 301

Also add a branded public/404.html (Cloudflare Pages serves it automatically for any unmatched path):

  <!doctype html><html lang="en"><head><meta charset="utf-8">
  <title>Page not found — Subsidy Dropoff</title>
  <link rel="stylesheet" href="/styles.css"></head><body><div class="wrap">
  <header><div class="brand"><span>Subsidy Dropoff</span></div></header>
  <main><h1>That page doesn't exist.</h1>
  <p class="lede">Try the <a href="/">subsidy cliff calculator</a>.</p></main>
  </div></body></html>
```

11 Default npm scripts (dev/build/start/lint) all launch the orphaned Next.js app, not the deployed site

MEDIUM QUICK WIN

package.json's unqualified scripts — “dev”, “build”, “start”, “lint” — all operate exclusively on the undeployed app/ tree. There is no script anywhere to preview the code that is actually live, so a new contributor running `npm run dev` sees an abandoned mockup with none of the current production copy or behavior.

VERBATIM FIX

```
Rename the four Next-specific scripts and add a real preview script for the deployed site (leave every other
existing script untouched):

  "scripts": {
    "dev:next": "next dev",
    "build:next": "next build",
    "start:next": "next start",
    "lint:next": "next lint",
    "serve": "npx --yes http-server public -p 8788 -c-1",
    ... (typecheck, engine:build, engine:typecheck, test, test:watch, etl, etl:verify, dev:engine unchanged)
  }
```

12 npm run lint is non-functional, and the deployed JS/CSS have zero lint or format gate

MEDIUM QUICK WIN

package.json declares “lint”: “next lint”, but no eslint or eslint-config-next is installed and no .eslintrc/eslint.config file exists anywhere — next lint fails or triggers an interactive install prompt that cannot succeed in CI, and deploy.yml never calls it anyway. Separately, no lint or format tool of any kind targets public/app.js or public/styles.css, the code that actually ships.

VERBATIM FIX

```
Install and configure eslint for the Next tree:

  npm install -D eslint eslint-config-next

  // .eslintrc.json
  { "extends": "next/core-web-vitals" }

Add a second, working gate for the deployed vanilla files in .github/workflows/deploy.yml's Verify job:

  - name: Format check (deployed assets)
    run: npx --yes prettier@3 --check "public/index.html" "public/app.js" "public/styles.css"
```

13 next.config.mjs suppresses TypeScript build errors on the tree meant to be the higher-quality stack

LOW QUICK WIN

next.config.mjs sets `typescript: { ignoreBuildErrors: true }`. The implicit case for eventually cutting over to the Next.js tree is that it is the more capable, better-tooled implementation — suppressing its own build-time type errors undermines that argument and lets real type errors ship silently to an eventual cutover.

VERBATIM FIX

```
// next.config.mjs
/** @type {import('next').NextConfig} */
const nextConfig = {}

export default nextConfig

(Remove the typescript.ignoreBuildErrors override entirely; fix whatever errors it was masking instead.)
```

14 Theme system never declares CSS color-scheme, so native form-control chrome ignores dark mode

LOW QUICK WIN

styles.css flips custom properties on prefers-color-scheme: dark — a clean, JS-free approach — but never sets the CSS color-scheme property. Without it, the browser doesn't know the page has a working dark palette, so the plan-year <select>'s own OS-native popup and scrollbars keep rendering in the OS's light style even once the page has switched to dark.

VERBATIM FIX

```
Add to styles.css's existing :root block, alongside the other root-level declarations:

  :root {
    color-scheme: light dark;
    --ink: #0b0d0f;
    --paper: #faf9f7;
    ...
  }

(No duplicate declaration needed inside the dark media block — `light dark` lets the browser auto-select per
the same prefers-color-scheme signal already driving the custom properties.)
```

Category C — SEO, Structured Data & Discoverability

> What a crawler and a social-share card actually see today.

15 og:image and Twitter Card tags missing site-wide despite og.png already existing at the wrong aspect ratio

HIGH QUICK WIN

public/index.html carries og:title/description/type/url but no og:image, and zero twitter:* tags exist anywhere in the live page. public/og.png (318KB) sits unreferenced in the same deploy — confirmed by direct inspection to be 1024×1024 (square), not the 1200×630 (1.91:1) ratio social platforms expect, so it would be auto-cropped unpredictably even once wired in. Every link share to Slack, iMessage, X, LinkedIn, or Facebook currently renders as a bare text link with no preview.

VERBATIM FIX

```
1) Re-export the social image at 1200×630 and compress under ~150KB (pngquant/oxipng), replacing public/og.png.
2) Add to the <head> of public/index.html, immediately after the existing og:url line:

  <meta property="og:image" content="https://subsidydropoff.com/og.png">
  <meta property="og:image:width" content="1200">
  <meta property="og:image:height" content="630">
  <meta name="twitter:card" content="summary_large_image">
  <meta name="twitter:title" content="Subsidy Dropoff">
  <meta name="twitter:description" content="Find the exact income at which your ACA health-insurance subsidy
disappears — and what crossing it costs.">
  <meta name="twitter:image" content="https://subsidydropoff.com/og.png">

3) Validate with the Twitter Card Validator, Facebook Sharing Debugger, and LinkedIn Post Inspector once live.
```

16 Live web search confirms zero indexation and zero backlinks anywhere on the web for the domain

HIGH QUICK WIN

A site-scoped search and a bare quoted-domain search (run live this session) both returned zero results referencing subsidydropoff.com anywhere — no indexed pages, no backlinks, no directory listings, no social mentions. This is at least as plausibly explained by the 22-day-frozen deploy (Issue 1) as by the metadata gaps in this category, since no fresh-content crawl signal has shipped in over three weeks regardless of what ships in source.

VERBATIM FIX

```
Once Issues 1, 17–19 are fixed and a deploy actually lands: manually submit
https://subsidydropoff.com/sitemap.xml in Google Search Console (Sitemaps report) and Bing Webmaster Tools, use
GSC's URL Inspection tool to “Request Indexing” on the homepage directly, and pursue a handful of backlinks —
starting with plain links from the other sibling portfolio properties (see Document 1, §9).
```

17 Zero JSON-LD structured data anywhere on the deployed page

MEDIUM QUICK WIN

No <script type="application/ld+json"> exists anywhere in public/. This forfeits rich-result eligibility (rich snippets, sitelinks) — an enhancement rather than a blocking defect, and a low-cost addition since the descriptive copy already exists in the page's own meta description.

VERBATIM FIX

```
Add before </head> in public/index.html:

  <script type="application/ld+json">
  {
    "@context": "https://schema.org",
    "@type": "WebApplication",
    "name": "Subsidy Dropoff",
    "url": "https://subsidydropoff.com/",
    "applicationCategory": "FinanceApplication",
    "operatingSystem": "Any",
    "description": "Find the exact household income at which your ACA health-insurance premium tax credit
disappears at 400% of the federal poverty line, and what crossing it costs.",
    "offers": { "@type": "Offer", "price": "0", "priceCurrency": "USD" }
  }
  </script>
```

18 robots.txt does not exist anywhere in public/

MEDIUM QUICK WIN

Confirmed by direct search: no robots.txt file exists. Its absence does not itself block crawling, but it removes the standard Sitemap: pointer and the ability to Disallow the orphaned /photos/ URLs (Issue 20) — a baseline file a Fortune-500 property ships at effectively zero cost.

VERBATIM FIX

```
Create public/robots.txt with exactly:

  User-agent: *
  Allow: /
  Disallow: /photos/

  Sitemap: https://subsidydropoff.com/sitemap.xml
```

19 sitemap.xml does not exist anywhere in public/

MEDIUM QUICK WIN

Confirmed by direct search: no sitemap.xml exists. For a single-URL site this is not required for crawling, but its absence removes a lastmod freshness signal and gives robots.txt nothing to point at — and becomes more valuable the moment new guide/methodology pages ship (Issue 42).

VERBATIM FIX

```
Create public/sitemap.xml with:

  <?xml version="1.0" encoding="UTF-8"?>
  <urlset xmlns="https://www.sitemaps.org/schemas/sitemap/0.9">
    <url>
      <loc>https://subsidydropoff.com/</loc>
      <lastmod>2026-08-09</lastmod>
      <changefreq>weekly</changefreq>
      <priority>1.0</priority>
    </url>
  </urlset>

Set <lastmod> to the date of the actual last successful deploy — currently 2026-08-09 per CI history — and add
a <url> entry for each new page (/methodology/, /guide/) as it ships.
```

20 10.4MB of unreferenced legacy images ship inside the production deploy artifact as orphaned, crawlable URLs

MEDIUM QUICK WIN

public/photos/ (six PNGs, 10.1MB) plus public/og.png (318KB) account for 10.4MB of the 11MB total public/ directory Cloudflare Pages deploys (94%). A direct grep confirms zero references to the six photos anywhere in the live HTML/JS/CSS — they are leftovers from the orphaned Next.js redesign, sitting at guessable, crawlable URLs with no relation to the live product.

VERBATIM FIX

```
git rm -r public/photos
git commit -m "chore: remove 10.1MB of unreferenced photos from Cloudflare Pages deploy artifact"

Add a CI guard so this cannot silently recur — insert before the wrangler deploy step in
.github/workflows/deploy.yml:

  - name: Fail build on unreferenced public/ assets
    run: |
      for f in $(find public -type f \( -name '*.png' -o -name '*.jpg' -o -name '*.jpeg' -o -name '*.svg' \));
do
        rel="/${f#public/}"
        grep -rqF "$rel" public/index.html public/app.js public/styles.css || { echo "Unreferenced asset: $f";
exit 1; }
      done
```

21 No dedicated PNG/ICO favicon or apple-touch-icon

LOW QUICK WIN

The only icon reference is an inline data:image/svg+xml URI. There is no 16×16/32×32 PNG fallback for crawlers and UIs that don't rasterize inline SVG favicons, and no apple-touch-icon for iOS home-screen presentation. (Cross-references the real icon set needed for PWA installability — Issue 39.)

VERBATIM FIX

```
Export the existing brand mark to PNGs and add to the <head> of public/index.html:

  <link rel="icon" type="image/png" sizes="32x32" href="/favicon-32.png">
  <link rel="icon" type="image/png" sizes="16x16" href="/favicon-16.png">
  <link rel="apple-touch-icon" sizes="180x180" href="/apple-touch-icon.png">
```

Category D — UX, Content Interaction & Accessibility

The submit-then-result flow that is the entire product.

22 The “ages” field's inputmode locks mobile users into a keypad that has no comma — the exact input it requires

HIGH QUICK WIN

`<input id="ages" inputmode="numeric" placeholder="60, 58" value="60, 58">` — the field's own placeholder demonstrates a comma-separated list is expected, but inputmode="numeric" hints a digits-only keypad that iOS/Android reliably render with no comma key and no keyboard-switch affordance. An unpunctuated entry is sent as one out-of-range age and rejected server-side, turning a keyboard limitation into a failed submission for every multi-person household — exactly the persona the page's own hero copy is built around.

VERBATIM FIX

```
In public/index.html, drop the numeric inputmode so the OS shows its default text keyboard (which has a comma
key):

  <input id="ages" name="ages" inputmode="text" autocomplete="off"
         placeholder="60, 58" value="60, 58" required>

Longer-term IA fix: replace the single delimited field with one number input per household member,
added/removed via a repeater bound to householdSize — but the one-line inputmode fix unblocks mobile entry
immediately.
```

23 Validation errors expose raw API field keys instead of field labels, and are never wired to the offending input

HIGH QUICK WIN

`err.textContent = data.problems.map(p => `${p.field}: ${p.message}`).join(" · ")` surfaces raw identifiers like householdSize and employerCoverage.employeeMonthlyContribution, not visible labels. No aria-invalid or aria-describedby links any input to its error, and focus never moves to the first invalid field.

VERBATIM FIX

```
In public/app.js, add a label map and per-field wiring in the validation-error branch of submit():

  const FIELD_LABELS = {
    zip: "ZIP code", planYear: "Plan year", householdSize: "Household size",
    income: "Expected income", ages: "Ages of people needing coverage", countyFips: "County",
  };

  if (data.error === "validation_failed") {
    out.innerHTML = "";
    form.querySelectorAll("[aria-invalid]").forEach((el) => el.removeAttribute("aria-invalid"));
    err.textContent = data.problems
      .map((p) => `${FIELD_LABELS[p.field] ?? p.field}: ${p.message}`)
      .join(" · ");
    let firstInvalid = null;
    for (const p of data.problems) {
      const input = form.elements.namedItem(p.field.split(".")[0]);
      if (input) {
        input.setAttribute("aria-invalid", "true");
        input.setAttribute("aria-describedby", "err");
        firstInvalid ??= input;
      }
    }
    firstInvalid?.focus();
  }
```

24 No focus or scroll management on any dynamically injected content — every submission risks stranding keyboard/AT users

HIGH QUICK WIN

A full read of public/app.js confirms there is not a single .focus() call and no tabindex anywhere in the deployed source. Results, unavailable-state notices, and validation errors are all written into the DOM with zero programmatic focus movement — this fires on every interaction, not an edge case, since submit-then-result is the entire product.

VERBATIM FIX

```
In public/index.html, give the output and error regions focus targets:

  <p class="err" id="err" role="alert" tabindex="-1"></p>
  ...
  <div id="out" aria-live="polite" tabindex="-1"></div>

In public/app.js, inside submit()'s try block, move focus after each render:

  if (data.error === "validation_failed") {
    err.textContent = ...; out.innerHTML = ""; err.focus();
  } else if (data.ok) {
    out.innerHTML = renderResult(data); renderProvenance(data); out.focus();
  } else if (data.unavailable) {
    out.innerHTML = renderUnavailable(data.unavailable); renderProvenance(data); out.focus();
  } else {
    err.textContent = data.message ?? "Something went wrong."; out.innerHTML = ""; err.focus();
  }
```

25 No navigation chrome anywhere, and the only methodology content is hidden until a user has already entered their income

MEDIUM QUICK WIN

The header contains only a logo mark — no nav, no links. The sole explanation of how the tool derives its numbers is a per-result <details> block rendered exclusively after a successful API call, meaning a first-time visitor to an unbranded, unindexed domain must already type in their income before seeing any derivation or sourcing to decide whether to trust the tool.

VERBATIM FIX

```
1) Add a lightweight in-page nav to the header:

  <nav aria-label="Page sections">
    <a href="#estimate">Estimate</a>
    <a href="#methodology">How this is calculated</a>
  </nav>

2) Add id="estimate" to the existing <h2>Estimate</h2>.
3) Add a static, always-visible methodology section before the footer:

  <section id="methodology">
    <h2>How this is calculated</h2>
    <p class="lede">We start from the same SLCSP benchmark premium and applicable-percentage table the IRS uses
on Form 8962, then find the exact income where the premium tax credit hits zero at 400% of the federal poverty
line. Every number below shows its source and publish date.</p>
  </section>
```

26 No skip-to-content link, despite the exact visually-hidden CSS utility a skip link needs already existing unused

MEDIUM QUICK WIN

body's first child is the page wrapper directly — a WCAG 2.4.1 (Bypass Blocks) gap any automated scanner will flag. styles.css already defines the .sr visually-hidden utility class a skip link needs, but it is never referenced anywhere. Real-world impact is muted today (no repeated nav to bypass) but disappears the moment Issue 25's nav ships.

VERBATIM FIX

```
Immediately after <body>, before the page wrapper:

  <a class="skip-link" href="#main-content">Skip to calculator</a>

Change <main> to <main id="main-content" tabindex="-1"> (required for the anchor to move keyboard focus, not
just scroll position).

In styles.css, add (do not reuse .sr, which stays hidden even when focused):

  .skip-link {
    position: absolute; left: 1rem; top: -3rem; z-index: 10;
    background: var(--teal); color: #fff; padding: 0.625rem 1rem;
    border-radius: 8px; text-decoration: none; transition: top 0.15s ease;
  }
  .skip-link:focus { top: 1rem; }
```

27 Plan-year selector is statically hardcoded and never reconciled with the live /api/health endpoint

MEDIUM QUICK WIN

index.html hardcodes 2026/2027 as options. /api/health already exposes supportedPlanYears, but app.js's only fetch call is to /api/estimate — a user who picks 2027 gets no upfront signal that CMS hasn't published it yet; they only discover it after a full round trip.

VERBATIM FIX

```
In public/app.js, add on module load:

  fetch("/api/health")
    .then((r) => r.json())
    .then((h) => {
      const supported = new Set(h.supportedPlanYears ?? []);
      for (const opt of $("planYear").options) {
        if (!supported.has(Number(opt.value))) {
          opt.textContent = `${opt.value} (not yet published)`;
        }
      }
    })
    .catch(() => {});

(Left selectable rather than disabled, since “not published” can change without a redeploy — but now signaled
before submit.)
```

28 County-disambiguation buttons are an ungrouped flex row, and once picked there is no way to change county without retyping the ZIP

MEDIUM QUICK WIN

The candidate-county buttons render as a bare list with no fieldset/legend or group role identifying them as one “choose your county” control, so assistive tech announces unrelated buttons rather than a grouped choice. Separately, once a county is picked, the only way to re-trigger disambiguation is deleting and retyping a ZIP digit.

VERBATIM FIX

```
1) Group the buttons semantically in public/app.js's ambiguous-zip case:

  return notice("Which county?", "...",
    `<div class="choices" role="group" aria-label="Choose your county">${buttons}</div>`);

2) Add a “change county” escape hatch to results:

  // in renderResult(), when a county was pinned:
  const changeCounty = $("countyFips").value
    ? `<p class="hint"><button type="button" id="changeCounty">Not your county? Choose again</button></p>`
    : "";

  // in the out click listener, above the existing data-fips branch:
  out.addEventListener("click", (event) => {
    if (event.target.closest("#changeCounty")) {
      $("countyFips").value = ""; void submit(); return;
    }
    const button = event.target.closest("[data-fips]");
    if (!button) return;
    $("countyFips").value = button.dataset.fips; void submit();
  });
```

29 novalidate suppresses all native HTML5 constraint validation with no JS equivalent

LOW QUICK WIN

`<form id="f" novalidate>` disables the browser's free, instant constraint-validation UI even though every field already carries real constraints (pattern, min, max, required). submit() never calls reportValidity() as a substitute, so a blank required field produces no feedback until a full network round trip resolves.

VERBATIM FIX

```
Drop novalidate to restore native validation for free:

  <form id="f">

(If novalidate must stay for custom bubble styling later, the minimal alternative is adding `if
(!form.reportValidity()) return;` as the first line inside submit(), right after `event?.preventDefault();`.)
```

30 Custom focus-visible ring is scoped to input/select/button only — links fall back to the unstyled default

LOW QUICK WIN

The focus-visible rule omits the anchor selector, so the HealthCare.gov link, the state-exchange handoff link, and citation links inside the arithmetic disclosure all receive the browser's default outline instead of the on-brand teal ring — a design-consistency gap, not a conformance failure.

VERBATIM FIX

```
Extend the existing selector list in styles.css from:

  input:focus-visible, select:focus-visible, button:focus-visible {

to:

  input:focus-visible, select:focus-visible, button:focus-visible, a:focus-visible {
```

31 Dynamically rendered result headings break the page's heading outline

LOW QUICK WIN

The static page goes h1 → h2 (“Estimate”) with no further h2 anywhere. Everything rendered into the results region — notice titles and each arithmetic step — uses h3 directly with no parent h2, flattening what should be a two-level outline into an ungrouped list for screen-reader heading navigation.

VERBATIM FIX

```
Add a labeling heading immediately before the output region:

  <h2 class="sr" id="results-heading">Results</h2>
  <div id="out" aria-live="polite" tabindex="-1" aria-labelledby="results-heading"></div>

(This also puts the existing but currently-unused .sr utility class to work.)
```

Category E — Security & Privacy

Header hardening and the missing policy page.

32 Missing Cross-Origin isolation headers — COOP and CORP absent, COEP also absent

MEDIUM QUICK WIN

public/_headers sets no Cross-Origin-Opener-Policy or Cross-Origin-Resource-Policy. Without COOP, the top-level document shares a browsing context group with any cross-origin window that opens it, exposing it to Spectre-class side-channel reads. Without CORP, /api/* and /data/* responses can be pulled cross-origin as no-cors subresources and probed via timing side channels — more consequential here because /api/estimate returns a per-request, income-derived computed value.

VERBATIM FIX

```
In public/_headers, add to the existing /* block:

  Cross-Origin-Opener-Policy: same-origin
  Cross-Origin-Resource-Policy: same-origin
  Cross-Origin-Embedder-Policy: require-corp

Safe to add now: a grep of the deployed page confirms zero cross-origin subresources are loaded. Re-check
before any future third-party embed (ad tag, widget) is added — that resource must serve its own CORP/COEP-
compatible headers or the embed will silently fail to load.
```

33 CSP has no violation-reporting endpoint — a future policy break, or a regression of the “zero third-party” promise, fails silently

MEDIUM QUICK WIN

The deployed CSP is otherwise strict (no unsafe-inline/unsafe-eval, matching a page with zero inline scripts), but with no reporting endpoint, a future change that introduces a blocked resource — or an accidentally reintroduced tracking script — fails silently in the browser with no signal to the operator.

VERBATIM FIX

```
1) In public/_headers, add a Reporting-Endpoints header and append report-to to the existing CSP line:

  Reporting-Endpoints: csp-endpoint="https://subsidydropoff.com/api/csp-report"
  Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data:;
connect-src 'self'; font-src 'self' data:; base-uri 'none'; form-action 'none'; frame-ancestors 'none'; object-
src 'none'; report-to csp-endpoint

2) Add functions/api/csp-report.ts (a static route Cloudflare Pages routes ahead of the [[route]].ts catch-
all):

  export const onRequestPost: PagesFunction = async ({ request }) => {
    try {
      const body = await request.text();
      console.log('csp-violation', body.slice(0, 4000));
    } catch {}
    return new Response(null, { status: 204, headers: { 'cache-control': 'no-store' } });
  };

Logs to Cloudflare's Functions log stream; persists nothing, consistent with the project's zero-persistence
design.
```

34 Permissions-Policy locks down only 3 of roughly 25 gatable browser features

LOW QUICK WIN

The current policy blocks geolocation, microphone, camera, and interest-cohort — the most sensitive APIs — but leaves payment, usb, serial, hid, bluetooth, display-capture, and others at browser default. A form-input-to-JSON calculator needs none of these.

VERBATIM FIX

```
Replace the Permissions-Policy line in public/_headers with:

  Permissions-Policy: accelerometer=(), ambient-light-sensor=(), autoplay=(), battery=(), bluetooth=(), camera=
(), display-capture=(), document-domain=(), encrypted-media=(), fullscreen=(), gamepad=(), geolocation=(),
gyroscope=(), hid=(), idle-detection=(), magnetometer=(), microphone=(), midi=(), payment=(), picture-in-
picture=(), publickey-credentials-get=(), screen-wake-lock=(), serial=(), sync-xhr=(), usb=(), web-share=(),
xr-spatial-tracking=(), interest-cohort=(), browsing-topics=()
```

35 CSP img-src allows any HTTPS origin, though the deployed page references zero external images

LOW QUICK WIN

img-src is set to 'self' data: https: — the data: grant is genuinely used by the inline SVG favicon, but a full grep shows no <img> tag, no CSS url(), and no JS-inserted image anywhere in the deployed code. The https: wildcard widens attack surface (e.g. beaconing to an attacker-chosen host if a markup-injection bug ever appears) for zero current benefit.

VERBATIM FIX

```
Tighten img-src in public/_headers by removing the unused https: source:

  Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data:;
connect-src 'self'; font-src 'self' data:; base-uri 'none'; form-action 'none'; frame-ancestors 'none'; object-
src 'none'

If an external image is intentionally added later, allowlist that specific host rather than reinstating the
wildcard.
```

36 No dedicated Privacy Policy or Terms of Use page exists anywhere on the deployed site

HIGH QUICK WIN

The only privacy-relevant text is a three-sentence footer statement. There is no privacy.html, terms.html, or linked legal document anywhere in public/ — no legal-entity identification, no stated contact channel, no disclosure of what the hosting/CDN layer processes. This blocks the most likely monetization channel outright: AdSense and most affiliate/referral partners require a linked privacy policy before onboarding.

VERBATIM FIX

```
Create public/privacy.html and public/terms.html, link them from the footer:

  <p class="legal-links">
    <a href="/privacy.html">Privacy Policy</a> · <a href="/terms.html">Terms of Use</a>
  </p>

Minimum viable privacy.html body:

  <h1>Privacy Policy</h1>
  <p><strong>Effective date:</strong> 2026-08-31</p>
  <p>Subsidy Dropoff (subsidydropoff.com) does not use cookies, does not run
  analytics or tracking scripts, and does not create accounts. The household
  size, ZIP code, ages, and income you enter are sent once to compute a result
  and are not logged, stored, or retained (every API response is served with
  Cache-Control: no-store).</p>
  <p>This site is hosted on Cloudflare Pages. As with any CDN, Cloudflare
  necessarily processes standard connection metadata (such as your IP address)
  to route the request and provide security/DDoS protection; this application
  does not access, log, or receive that data separately. See
  <a href="https://www.cloudflare.com/privacypolicy/" rel="noopener">Cloudflare's Privacy Policy</a>.</p>
  <p>Questions: <a href="mailto:privacy@subsidydropoff.com">privacy@subsidydropoff.com</a></p>
```

37 Privacy claim omits the CDN/edge-layer processing performed by Cloudflare itself

MEDIUM QUICK WIN

The footer states “We collect nothing” — accurate for the application code, but incomplete as a privacy statement, since Cloudflare necessarily processes connection-level metadata for every request to route traffic and provide DDoS protection. An unqualified “we collect nothing” is an overclaim a careful reader or regulator could challenge.

VERBATIM FIX

```
Add one sentence to the footer directly after the existing collection statement:

  <p>
    We collect nothing. No account, no email, no analytics on your inputs. Your
    income figure is used to compute a number and then discarded. This site is
    hosted on Cloudflare Pages, which processes standard connection metadata
    (e.g. your IP address) to serve the page and prevent abuse; this application
    does not access or store that data. See <a href="/privacy.html">our Privacy Policy</a>.
  </p>
```

38 No operator/legal-entity identity or contact method anywhere on the live site

MEDIUM QUICK WIN

No file in public/ names a responsible individual, company, or contact email. Combined with Issue 36, a user or regulator has no way to identify who operates the tool or how to reach them about a compliance concern — a baseline expectation for any site touching health/financial inputs.

VERBATIM FIX

```
Add a contact/identity line to the footer:

  <p class="legal-links">
    Subsidy Dropoff · <a href="mailto:privacy@subsidydropoff.com">privacy@subsidydropoff.com</a>
  </p>

(Replace with the operator's real monitored address before shipping.)
```

Category F — PWA, Content Depth & Monetization Readiness

Installability, the content gap, and forward-looking guardrails.

39 No web app manifest and no real icon files anywhere — site fails every PWA installability check

HIGH LONGER-TERM

No manifest.json exists, and index.html's only icon reference is an inline 32×32 SVG data URI — no apple-touch-icon, no 192/512px PNGs, no maskable icon. Chrome/Edge will never offer an install prompt, and iOS home-screen saves fall back to an auto-generated page screenshot instead of the brand mark.

VERBATIM FIX

```
Create public/manifest.json:

  {
    "name": "Subsidy Dropoff",
    "short_name": "Subsidy Dropoff",
    "description": "Find the exact income where your ACA health-insurance subsidy disappears.",
    "start_url": "/", "scope": "/", "display": "standalone",
    "background_color": "#faf9f7", "theme_color": "#0e7c7b",
    "icons": [
      { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png", "purpose": "any" },
      { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png", "purpose": "any" },
      { "src": "/icons/icon-512-maskable.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable" }
    ]
  }

Add to <head> after the existing icon link: `<link rel="manifest" href="/manifest.json">` and `<link
rel="apple-touch-icon" href="/icons/apple-touch-icon.png">`. Export the existing header brand mark (the teal
path shape already in index.html) to the PNG sizes above — background_color/theme_color are taken directly from
the site's own --paper/--teal tokens. No CSP change needed.
```

40 No theme-color meta tag

MEDIUM QUICK WIN

No <meta name="theme-color"> exists. Android Chrome and any future PWA install will use a default status-bar color instead of the site's own light/dark backgrounds — a regression versus the undeployed Next.js tree, which already declares the correct pairing.

VERBATIM FIX

```
Add to <head>, after the description meta tag:

  <meta name="theme-color" content="#faf9f7" media="(prefers-color-scheme: light)">
  <meta name="theme-color" content="#0b0d0f" media="(prefers-color-scheme: dark)">
```

41 No service worker — zero offline resilience despite an architecture built to make it cheap

MEDIUM LONGER-TERM

submit() does a bare fetch with no cache fallback and no offline page; a connectivity blip surfaces only a raw error string. This is unusually cheap to close here because /data/* is already marked immutable for a year in public/_headers — the benchmark shards are already designed to be precached.

VERBATIM FIX

Create public/sw.js:

```
const CACHE = "subsidy-dropoff-v1";
const PRECACHE = ["/", "/styles.css", "/app.js", "/manifest.json"];
self.addEventListener("install", (event) => {
  event.waitUntil(caches.open(CACHE).then((c) => c.addAll(PRECACHE)));
  self.skipWaiting();
});
self.addEventListener("activate", (event) => {
  event.waitUntil(caches.keys().then((keys) =>
    Promise.all(keys.filter((k) => k !== CACHE).map((k) => caches.delete(k)))));
  self.clients.claim();
});
self.addEventListener("fetch", (event) => {
  const url = new URL(event.request.url);
  if (url.pathname.startsWith("/data/")) {
    event.respondWith(caches.match(event.request).then((cached) =>
      cached || fetch(event.request).then((res) => {
        const copy = res.clone();
        caches.open(CACHE).then((c) => c.put(event.request, copy));
        return res;
      })));
    return;
  }
  if (event.request.mode === "navigate") {
    event.respondWith(fetch(event.request).catch(() => caches.match("/")));
  }
});
```

Append to the bottom of public/app.js (not an inline script — CSP's script-src 'self' with no unsafe-inline would block that):

```
if ("serviceWorker" in navigator) {
  window.addEventListener("load", () => navigator.serviceWorker.register("/sw.js"));
}
```

42 Entire public content footprint is one page under 200 words — no guide, blog, FAQ, or methodology content ships anywhere

CRITICAL LONGER-TERM

public/index.html contains exactly two headings and roughly 189 words of prose total. There is no blog, guide, FAQ, or live /methodology route. Meanwhile roughly 1,000–1,150 words of genuinely well-written, differentiated subject-matter explanation already exist — the “two ways the credit ends” distinction and Rev. Proc. 2026-26 sourcing in README.md, and a largely complete methodology page in the undeployed Next.js tree — with zero public-facing equivalent. Named competitors acacalc.com and taxcliffs.co both pair a calculator with exactly this kind of guide content.

VERBATIM FIX

```
Since production is plain static HTML (no Next.js build step runs in CI), hand-port the prose — strip the
JSX/React, keep the explanatory text — into two new static pages picked up automatically by the existing
`wrangler pages deploy public` step, no CI change required:

  public/methodology/index.html  — ported from app/methodology/page.tsx
  public/guide/index.html        — ported from README.md's “Two ways the credit ends” section (lines 68-93),
with:
    <title>Not everyone has a subsidy cliff — how the ACA premium tax credit actually phases out</title>
    <meta name="description" content="Some households lose their ACA subsidy gradually as income rises. Others
keep the full credit right up to 400% of the poverty line, then lose it all at once. Here's how to tell which
applies to you.">

Link both from the homepage nav (Issue 25) and add both to sitemap.xml (Issue 19). See the full 30-Day Plan for
the complete four-week content sequence.
```

43 No FAQ content despite the tool directly answering the exact questions competitor guides are built around

LOW LONGER-TERM

Competitor content (e.g. taxcliffs.co's guide) is structured around standalone questions this product already has crisp, accurate internal answers to — none of it is surfaced as user-facing Q&A or paired with FAQPage schema for rich-result eligibility.

VERBATIM FIX

```
Add before the footer:

  <h2>Frequently asked questions</h2>
  <dl>
    <dt>What is the ACA subsidy cliff?</dt>
    <dd>At 400% of the federal poverty line, the premium tax credit does not taper off — it disappears
entirely. One dollar of income above the line drops the credit from its full amount to zero.</dd>
    <dt>Why does this matter more starting in 2026?</dt>
    <dd>The enhanced premium tax credits from the American Rescue Plan and Inflation Reduction Act expired 31
December 2025 and were not extended, reinstating the hard 400% cliff for plan years 2026 and 2027.</dd>
  </dl>

Pair with FAQPage JSON-LD using the same question/answer text.
```

44 No consent-management scaffolding exists for the cookies future monetization would introduce

LOW LONGER-TERM

Zero cookies are set today — correctly, no consent banner exists, since one now would be compliance theater. But the moment an ad network or affiliate pixel is added, cookies or fingerprinting will likely follow, triggering CCPA “Do Not Sell or Share” disclosure obligations for a self-selected, income-disclosing visitor base.

VERBATIM FIX

```
No code change needed pre-monetization. Add as an explicit checklist item on the first monetization PR: it must
also ship (1) a minimal self-hosted consent gate that runs before any third-party script executes, and (2) a
footer link `<a href="#" id="dns">Do Not Sell or Share My Personal Information</a>` wired to that gate, per
Cal. Civ. Code §1798.135 — added in the same PR as the first tracking script, never retrofitted after.
```

45 No scaffold exists for disclosing broker licensure — the precondition the README's own strictest monetization rule depends on

LOW LONGER-TERM

README states verbatim: “No compensation contingent on a sale while unlicensed” — the direct precondition for licensed-broker referral, the highest-value monetization channel in scope. Nothing in the codebase represents licensing status anywhere, unlike the disclaimer pattern, which is already duplicated in both the HTML and the API.

VERBATIM FIX

```
Pre-build the pattern now, defaulted off, mirroring the existing DISCLAIMER-duplication approach in
src/api/handler.ts:

  export const BROKER_LICENSING_STATUS = {
    licensed: false,
    npn: null as string | null,
    licensedStates: [] as string[],
  };

Gate any future broker-referral CTA or API field on BROKER_LICENSING_STATUS.licensed only, so enabling broker
referral in a state becomes a data edit rather than new conditional code written under launch pressure.
```

46 README's “no tracking / no PII” rules are documented policy but not enforced by CI

LOW QUICK WIN

The project's well-reasoned non-negotiable build rules are currently enforced only by convention and code review — none of the 117 vitest tests assert that the deployed files stay free of tracking scripts, cookies, or third-party calls. A future contributor (or an AI-assisted PR) could reintroduce a tracking pixel with nothing catching it before deploy.

VERBATIM FIX

```
Add test/no-tracking.test.ts:

  import { readFileSync } from "node:fs";
  import { describe, expect, it } from "vitest";

  describe("privacy invariants", () => {
    const html = readFileSync("public/index.html", "utf8");
    const js = readFileSync("public/app.js", "utf8");

    it("ships no known tracking/analytics scripts", () => {
      const bannedPatterns = [
        /gtag\(/i, /googletagmanager/i, /google-analytics/i,
        /facebook\.net\/.*\/fbevents/i, /connect\.facebook\.net/i,
        /plausible\.io/i, /umami/i, /hotjar/i, /segment\.com/i, /mixpanel/i,
      ];
      for (const pattern of bannedPatterns) {
        expect(html).not.toMatch(pattern);
        expect(js).not.toMatch(pattern);
      }
    });

    it("sets no document.cookie anywhere in the shipped client bundle", () => {
      expect(js).not.toMatch(/document\.cookie\s*=/);
    });
  });
```

COMPETITIVE LANDSCAPE

19 Competitors, Full Detail

The 2026 expiration of enhanced premium tax credits triggered a genuine content gold rush in this niche. Live search this session surfaced well over 20 active sites/

once sub-entries are counted; the 19 rows below carry full positioning, content-strategy, and gap detail. See Document 1 §8 for the three-tier summary.

COMPETITOR	POSITIONING	CONTENT STRATEGY	GAP VS. SUBSIDY DROPOFF
healthinsurance.org — Obamacare Subsidy Calculator https://www.healthinsurance.org/obamacare/subsidy-calculator/	Legacy (est. 1994-lineage) health-reform editorial authority; the calculator is one feature bolted onto a massive, journalist-maintained ACA policy publication cited by outlets like CNBC.	Calculator + huge surrounding editorial library: per-state subsidy/marketplace explainers, 'will you receive a subsidy' FAQ pages, annually updated policy analysis (2026 cliff-return coverage already live).	Impossible to out-authority head-on, but its calculator is buried inside a sprawling info-site — subsidydropoff can win on single-purpose speed/focus/privacy IF it ships even a fraction of the explainer depth this site has (it currently has none).
National Tax Tools — Premium Tax Credit Calculator https://nationaltaxtools.com/calculators/premium-tax-credit-calculator/	Tax-preparer-framed calculator (Form 8962, SLCSP, MAGI) inside a broad free multi-calculator suite (AGI/MAGI, Roth conversion, refund tracker, payroll).	Calculator + a dedicated companion guide page ('Premium Tax Credit Guide 2026: 400% FPL Cliff, Form 8962, MAGI, SLCSP') plus a full site map of dozens of related tax calculators/guides.	Its guide page models exactly the tax-mechanics content (Form 8962, SLCSP, MAGI) subsidydropoff has zero of; broad-suite breadth dilutes its topical authority per calculator, which subsidydropoff's single-purpose focus could beat if paired with equivalent depth.
ACA Subsidy Guide https://acasubsidyguide.com/calculator	Subsidy-cliff-specific brand with a name almost identical in concept to subsidydropoff.com; leans into actionable 'stay under the cliff' income-strategy advice.	Calculator + programmatic all-50-state calculator pages (e.g., /states/indiana, /states/alaska) + blog content ('5 Legal Ways to Lower Your Income and Keep Your ACA Subsidy', an /income-strategies page covering HSA/IRA/401(k) MAGI-reduction tactics).	Has a massive programmatic-SEO footprint (50 state pages + strategy blog) that subsidydropoff entirely lacks, and is the closest brand/positioning collision in the set — a direct threat for the exact 'subsidy cliff calculator' query cluster.
TaxPayers.net — ACA Subsidy Cliff Calculator https://taxpayers.net/calculators/aca-subsidy-cliff	Part of an 89-calculator/122-guide free tax-tool suite; differentiates on trust/accuracy rather than UX polish.	Calculator explicitly covers 2026 nuance (year-aware repayment exposure after OBBB repealed repayment caps) plus a publicly stated 'verification log' cross-checking figures against primary IRS sources.	The public verification-log/primary-source-citation transparency is a trust signal subsidydropoff doesn't visibly offer; also covers more advanced 2026-specific nuance (OBBB repayment-cap repeal) than a bare calculator would.

COMPETITOR	POSITIONING	CONTENT STRATEGY	GAP VS. SUBSIDY DROPOFF
ACA Calc https://acacalc.com/	Calculator-first branding (name is literally 'ACA calc') with a dedicated cliff explainer as its flagship content piece.	Calculator + /guides/2026-subsidy-cliff, which precisely explains the mechanics (400.00% FPL still gets a credit, 400.01% gets $0) and FPL-by-household-size math state by state.	The single closest structural analog to subsidydropoff (one calculator + one authoritative cliff guide) — this is the direct head-to-head competitor for the core query, and its guide is exactly the content layer subsidydropoff is missing.
TaxCliffs / ACA Subsidy Cliff (related domains) https://taxcliffs.co/learn/aca-subsidy-cliff-2026-guide (also serving from acasubsidycliff.com)	Positions the ACA cliff as one of several interconnected retirement 'income cliffs' (ACA, IRMAA, Social Security torpedo, capital-gains bump zone, NIIT, Roth conversions, widow's-tax penalty) rather than a standalone topic.	7 free calculators + a 'Learn' hub with state pages (e.g., /states/new-york) and adjacent guides (ACA open enrollment 2027 dates). Heavy topical-cluster SEO targeting the same early-retiree/self-employed persona from multiple angles.	Bundles the ACA cliff into a full suite of adjacent retirement-income cliffs the same user persona hits (IRMAA, NIIT, Roth) — a visitor solving 'ACA cliff' here gets cross-sold five more calculators in one session; subsidydropoff is single-purpose and captures none of that adjacent search demand.
QuantCalc — ACA Subsidy Cliff Calculator https://quantcalc.app/blog/aca-subsidy-cliff-calculator-free-tool/ (tool at quantcalc.app/aca/)	A retirement-planning SaaS (Monte Carlo simulator, $49–99 lifetime PRO tier) that offers the ACA cliff calculator as a free top-of-funnel tool inside a much larger content-marketing engine.	Blog-driven growth: comparison posts ('I Tested 6 Retirement Calculators', 'Best Monte Carlo Retirement Calculators Compared'), a public methodology page, and multiple ACA-adjacent posts (capital gains vs. the cliff, Obamacare calculator explainer).	This is the most direct threat to subsidydropoff's core claim: QuantCalc already contests the exact 'privacy-first, client-side, zero data collection' positioning, backed by a far larger comparison-content/SEO engine and a real monetization path — subsidydropoff has the engineering claim but none of the content to make it discoverable or credible against this competitor.

COMPETITOR	POSITIONING	CONTENT STRATEGY	GAP VS. SUBSIDY DROPOFF
ValuePenguin (LendingTree) — ACA Subsidy Calculator https://www.valuepenguin.com/aca-subsidy-calculator	Personal-finance comparison-shopping brand owned by publicly traded lead-gen company LendingTree; calculator is a content/lead asset inside a much larger financial-product marketplace.	Calculator plus companion explainer articles (e.g., 'What Is the Premium Tax Credit for Health Insurance?'); maintains a formal advertiser-disclosure page.	Represents the highest-domain-authority, most overtly commercial end of the field — subsidydropoff's genuine no-ads/no-affiliate stance is a real trust differentiator against this class of competitor, and should be messaged explicitly since ValuePenguin will likely outrank on generic 'ACA subsidy calculator' terms.
HealthcareInsider.com — ACA/Obamacare Subsidy Calculator https://healthcareinsider.com/aca-subsidy-calculator	Calculator embedded in a licensed insurance agency's marketing site (Healthcare.com Insurance Services, LLC).	Calculator + full ACA-plans content hub (best individual plans, how-to-calculate guides, 'will you qualify' explainers).	Clear commercial bias (calculator exists to generate insurance-agent leads); subsidydropoff's neutrality is a stronger trust position, though this competitor likely wins on buying-intent keywords where users actively want an agent.
Carrier/brokerage-owned calculators (Medical Mutual, myhealthinsurance.com) https://www.medmutual.com/Individuals-and-Families/Tax-Subsidy-Estimator ; https://www.myhealthinsurance.com/obamacare-calculator/	An insurance carrier's own 'Tax Subsidy Estimator' and an insurance brokerage's 'ACA Subsidy Calculator' — both pure lead-generation tools for a specific seller, not neutral estimators.	Thin — calculator plus minimal supporting copy; not designed as an SEO content hub.	Low content depth and obvious bias make these weak on trust, but their brand recognition and existing customer relationships still pull traffic subsidydropoff can't intercept without its own content/SEO presence.
KFF Health Insurance Marketplace Calculator / Beyond the Basics (CBPP) https://www.kff.org/interactive/subsidy-calculator/ ; https://www.healthreformbeyondthebasics.org/kaiser-family-foundation-subsidy-calculator/	The highest-trust nonprofit/policy-research brand in the space (Kaiser Family Foundation), with a CBPP-affiliated companion calculator built for enrollment counselors/navigators.	General (non-'cliff'-branded) subsidy calculator embedded in KFF's broader health-policy research output; Beyond the Basics targets professional navigators rather than consumers.	Same zero-monetization ethos as subsidydropoff, but an older/less polished interactive-tool UX and no 'subsidy cliff' news-hook framing or 2026-specific SEO content — subsidydropoff's modern single-page engineering is a real edge here if it's ever surfaced by content/SEO.

COMPETITOR	POSITIONING	CONTENT STRATEGY	GAP VS. SUBSIDY DROPOFF
PolicyEngine — ACA-Calc https://www.policyengine.org/us/aca-calc	Open-source policy nonprofit; the ACA cliff calculator is a thin front-end over a rigorous, published, peer-reviewed microsimulation model (PolicyEngine Core, covering 55+ federal/state/local programs) that also models the pre-2021 vs. enhanced PTC schedules.	Research-blog announcement post ('Introducing ACA-Calc') plus open-source code/methodology on GitHub and a JOSS-published paper — content aimed at researchers/journalists, not consumer SEO.	The most technically credible neutral competitor in the set; if subsidydropoff wants to claim calculation accuracy, PolicyEngine's published, peer-reviewed methodology is the bar to match or cite against — subsidydropoff currently publishes no methodology at all.
IRS Taxpayer Advocate Service — Premium Tax Credit Change Estimator https://www.taxpayeradvocate.irs.gov/estimator/premiumtaxcreditchange/	Official U.S. government tool; zero monetization, maximal authority, minimal design investment.	Narrow, utilitarian estimator for mid-year PTC changes only — no cliff framing, no guides.	Not a real UX competitor (dated, narrow, government-form-like), but it's the authoritative citation source every other competitor references — subsidydropoff should cite it explicitly to borrow credibility since it currently cites nothing.
PovertyLevelCalculator.com — ACA Calculator https://povertylevelcalculator.com/aca-calculator/	FPL-calculator suite brand that explicitly markets 'No Email Signup Required' on its ACA calculator page — competing directly on the same privacy-adjacent claim subsidydropoff wants to own.	Calculator + full FPL reference library (poverty-level tables by year/state, affordability calculator, dedicated 2026 subsidy page).	Exposes the real opening: 'no signup' has already become a common claim in this niche even among ad/affiliate-monetized sites, so subsidydropoff needs to differentiate more sharply — 'zero PII AND zero ads AND zero affiliate' is a strictly stronger, still-unclaimed position, but only if it's stated explicitly, which it currently isn't.
ACASubsidyCalculator.com https://acasubsidycalculator.com/	Standalone tool built specifically around the 'no email, no name, no phone required' promise, claiming to be 'built by experienced developers and health insurance professionals.'	Calculator + blog (e.g., 'How Are ACA Subsidies Calculated for 2026? (Complete Guide)', 'Try It Free Online').	The closest direct name/positioning collision on the 'no signup, privacy-respecting calculator' claim specifically — it already has the blog content depth subsidydropoff lacks, undercutting any first-mover advantage on the privacy message.

COMPETITOR	POSITIONING	CONTENT STRATEGY	GAP VS. SUBSIDY DROPOFF
Coast Retirement — ACA Subsidy Cliff Calculator https://coastretirement.com/aca-cliff	FIRE/early-retirement planning content site; the ACA cliff tool is one of several retirement calculators (FIRE number, retire-at-50, drawdown optimizer) aimed at the early-retiree persona.	Interactive drag-slider UX ('drag your income to see exactly when you hit the cliff') built on Python-powered planner-grade models, plus narrative 'scenario stories' (real-person what-if case studies) as content marketing.	Sets a materially higher UX bar with an interactive drag-to-see-the-cliff visualization plus storytelling content — subsidydropoff's 'excellent backend engineering' needs a comparably compelling front-end/interaction layer and narrative content to compete on experience, not just accuracy.
CoverageMath.com — ACA Subsidy Calculator https://coveragemath.com/aca-subsidy-calculator/	Neutral-sounding, math-first branding; thin public footprint suggests a newer 2026-vintage entrant.	Calculator + guide framing similar to acacalc.com's model (limited detail found beyond title/meta).	Its very existence (a brand-new, thinly documented entrant with the same calculator+guide template as everyone else) is itself the signal: this niche is attracting a wave of new sites in 2026 specifically because of the cliff's return, which raises the competitive urgency for subsidydropoff to ship content now rather than stay frozen.
Calcix — ACA Subsidy & Early Retirement Health Insurance Calculator https://calcix.net/calculators/retirement-pension/early-retirement-health-insurance-aca-subsidy-cliff	General-purpose multi-category calculator site (retirement/pension vertical); the ACA cliff tool is one of dozens of unrelated calculators.	Minimal — thin single-calculator page inside a broad calculator directory.	Low differentiation and shallow content mean it's not a serious authority threat, but it adds to the sheer number of results crowding the SERP for the same queries subsidydropoff needs to rank for.
Insurance-agent / advisor content-marketing cluster e.g. life143.com, ratequote.com, retirementresearcher.com, premierhsllc.com, healthyfp.com, ungrindfi.com	Guide-only (mostly no calculator) blog posts from insurance agencies, financial-advisory firms, and FIRE/FI bloggers using '2026 ACA subsidy cliff' as a topical hook to drive advisory or insurance leads (e.g. 'A Mid-Year Coverage Playbook', 'A Planning Guide for Early Retirees').	Long-form explainer/guide content only — no interactive tool; built to rank for informational 'aca subsidy cliff 2026' queries and convert readers into consultation leads.	Collectively these sites outnumber and outrank on the pure informational/explainer side of the query set even without offering a calculator at all — proof that content depth alone (no tool required) is competing successfully for the same keywords subsidydropoff's calculator wants to own.

LANDSCAPE SUMMARY

The "ACA subsidy cliff" calculator niche is unexpectedly crowded for 2026: the enhanced-PTC expiration and the return of the hard 400% FPL cliff on 1/1/2026 triggered a content gold rush, and WebSearch surfaced well over 20 live, actively-updated competitors beyond the 8 seeds (ValuePenguin/LendingTree, healthcareinsider.com, KFF, PolicyEngine's open-source ACA-Calc, the IRS Taxpayer Advocate estimator, Medical Mutual and myhealthinsurance.com's carrier/brokerage tools, acasubsidycliff.com, povertylevelcalculator.com, acasubsidycalculator.com, coastretirement.com, calcix.net, coveragemath.com, plus a cluster of advisor/agent guide-only content sites). Three clear tiers emerged: (1) legacy health-content publishers and lead-gen brokerages (healthinsurance.org, ValuePenguin, healthcareinsider.com, carrier/brokerage tools) — highest domain authority and traffic, but the calculator is subordinate to an ads/affiliate/insurance-lead business model; (2) a programmatic-SEO "tax/retirement tool network" (nationaltaxtools.com, taxpayers.net, acasubsidyguide.com, taxcliffs.co/acasubsidycliff.com, acacalc.com, quantcalc.app, coastretirement.com, povertylevelcalculator.com, acasubsidycalculator.com, coveragemath.com, calcix.net) — dozens of near-identical calculator+guide+state-page sites racing the same "ACA subsidy cliff 2026" keyword set, most bundled into larger multi-calculator suites covering adjacent retirement-income cliffs (IRMAA, NIIT, Roth conversions), monetized via ads/affiliate/freemium upsell (QuantCalc's paid tier is the clearest example); and (3) nonprofit/policy-grade tools (KFF, PolicyEngine, CBPP's Beyond the Basics, the IRS estimator) — genuinely neutral and zero-monetization like subsidydropoff, but not "cliff"-branded, not SEO-optimized for the 2026 news moment, and running dated interactive-tool UX. Critically, subsidydropoff.com's core intended differentiator — "privacy-first, no signup" — is already contested messaging: QuantCalc explicitly claims client-side, browser-only, no-data-leaves-device calculation; povertylevelcalculator.com and acasubsidycalculator.com both market "no signup" front-and-center — yet the search evidence shows povertylevelcalculator.com still discloses ad/affiliate monetization behind that claim, meaning subsidydropoff's true zero-ads-zero-affiliate-zero-PII combination is a strictly stronger, still-unclaimed position, but only if stated explicitly. A direct web search for "subsidydropoff.com" returned no relevant results at all, corroborating the brief's description of a stale/unindexed deploy: every single competitor found, from the thinnest calculator directory to the most rigorous nonprofit model, has at minimum a companion explainer page, and most have full guide hubs, state-specific pages, MAGI-reduction strategy content, or methodology/verification disclosures — content layers subsidydropoff currently has none of. The practical gap is not engineering (subsidydropoff's zero-PII single-page architecture is competitive with or better than most of this field) but discoverability and credibility: no meta/OG tags, no cliff-mechanics guide, no cited methodology or IRS/primary-source references, no state-level or persona-specific (early-retiree, self-employed, HSA/IRA strategy) content, and a frozen deploy in a niche where competitors are actively publishing new comparison and update posts through August 2026.

MONETIZATION MAP

Monetization & Affiliate Opportunity Map

Nine channels, sequenced from lowest-risk to highest-risk, each with an explicit compliance note grounded in the project's own non-negotiable README rules (no compensation contingent on a sale while unlicensed; nothing that could mislead a consumer into thinking this is HealthCare.gov).

PORTFOLIO FIT

Across the portfolio, monetization intensity should scale inversely with regulatory sensitivity: the recipes/food site can run heavy affiliate + display since it carries no licensing risk, sports analytics can lean into ads/subscriptions, and internal ops tools aren't consumer-monetized at all — subsidydropoff sits at the strict end of that spectrum alongside the "health tools" bucket, closer in posture to nonprofit/policy players (KFF, PolicyEngine) than to the ad-heavy legacy publishers or lead-gen brokerages in its competitive set. Its monetization ceiling is intentionally lower and slower: trust and topical authority (the content gap competitors already exploit) have to be built before any revenue layer is added, and revenue must stay strictly non-contingent and off the calculator surface itself. What the portfolio structure buys it is cross-subsidy — portfolio-level cross-promotion (item 3) lets subsidydropoff borrow discovery traffic from the food/sports properties without importing their tracking stacks, while the site's genuine zero-PII/zero-ads-on-the-tool stance becomes a differentiated brand asset the portfolio can point to elsewhere ("the strict-privacy sibling site") rather than a liability. In short: this site should be the portfolio's slow-burn, trust-first outlier, not asked to match the monetization density of its less-regulated siblings.

Editorial guide & methodology content (cliff mechanics, Form 8962/MAGI, state FPL tables, cited to IRS/KFF/PolicyEngine)

QUICK WIN

Build the companion content layer every competitor already has: a canonical '2026 ACA Subsidy Cliff' guide, a methodology/verification page citing primary IRS figures, KFF, and PolicyEngine's published model, plus a short FAQ. This isn't monetization by itself — it's the prerequisite inventory (traffic, dwell time, topical authority) every channel below depends on, and it's the place to state the zero-PII/zero-ads posture explicitly, since competitors like povertylevelcalculator.com already market a 'no signup' claim while still running ads/affiliates behind it.

COMPLIANCE NOTE

Guides stay descriptive/educational only — no 'you qualify for $X' language; every dollar figure carries an 'estimate, not a determination' caveat with a link to HealthCare.gov or the relevant state exchange for the actual determination; every guide page carries an unmistakable 'not affiliated with HealthCare.gov or any government agency' footer, since several competitors already lean on .gov-adjacent framing.

Reader-funded support link (one-time/recurring donation via Stripe Payment Link or similar)

QUICK WIN

Add a plain 'support this calculator' donation link on the calculator page itself, styled like Wikipedia/PolicyEngine's nonprofit posture rather than a paywall or gate. The payment processor handles the transaction entirely off-site; no account or stored payment data touches subsidydropoff.

COMPLIANCE NOTE

Not insurance compensation of any kind, so the broker-licensing constraint is not implicated; implement as a plain outbound link rather than an embedded checkout iframe so no third-party cookies are set on the calculator page, preserving its zero-cookie claim; disclose in the privacy policy that clicking it leaves the site.

Portfolio cross-promotion module (static links to sibling niche-utility sites)

QUICK WIN

Add a lightweight 'more free tools' footer/nav module — plain links, no tracking pixels, no shared cookies or ad network — pointing to genuinely relevant sibling health tools (e.g., an HSA or MAGI calculator) and, more loosely, to the rest of the portfolio for brand discovery.

COMPLIANCE NOTE

Link-only, no shared analytics/cookies, so the calculator's zero-PII/zero-cookie claim stays intact; each linked sibling site must carry its own privacy policy and subsidydropoff should note that leaving the site means leaving its zero-data-collection environment (e.g., the food/recipe site may run affiliate cookies); never cross-promote anything insurance-sales-flavored from this surface, since that would blur the 'not an insurance seller' line the whole site depends on.

Contextual display advertising, scoped to future guide/editorial pages only — never the calculator

LONGER-TERM

Once the guide content (item 1) has traffic, run a contextual ad network (starting with a non-behavioral/contextual-only network, graduating to a premium programmatic network once traffic thresholds qualify) on guide, methodology, and FAQ pages exclusively. The calculator itself stays ad-free and cookie-free — the strongest version of the 'genuinely zero-PII, zero-ads' claim the competitive research says nobody in this niche is making explicitly.

COMPLIANCE NOTE

Flat CPM/CPC ad revenue is not compensation contingent on a sale, so it does not trigger insurance-broker licensing; ad creatives must be vetted to exclude insurance-agent lead-gen units ('get a free insurance quote') and anything resembling a government seal or HealthCare.gov branding; if the chosen network sets cookies, that is a narrow, clearly-disclosed exception limited to guide pages only, stated in-page ('guide pages may use advertising cookies; the calculator itself never does').

Non-insurance financial-affiliate links inside guide content (tax software, HSA/IRA providers, fee-only advisor directories)

LONGER-TERM

Inside the MAGI-reduction / 'stay under the cliff' guide content, link to HSA custodians, IRA providers, and tax-filing software (Form 8962 context) via standard affiliate programs, and to fee-only, flat-fee financial-planner directories (e.g., NAPFA/Garrett-style networks) rather than commission-based insurance agents.

COMPLIANCE NOTE

These are financial/tax products, not insurance policies, so affiliate commissions here do not touch the 'no compensation contingent on a sale while unlicensed as broker' constraint — but any advisor directory listed must be genuinely fee-only/non-insurance-commission to stay outside that line; every affiliate link carries an FTC-compliant disclosure; links live only inside editorial guide pages, never on the calculator's result screen, so a user is never nudged toward a paid product at the moment a subsidy number is shown.

Licensed-broker referral partnership, structured as a flat, non-contingent sponsorship LONGER-TERM

Offer a single, clearly labeled 'Sponsored' placement (e.g., 'Need help enrolling? [Partner], a state-licensed broker, sponsors this guide') paid as a flat recurring sponsorship fee unrelated to conversion — not a per-lead or per-sale referral fee — to a vetted, actually-licensed brokerage.

COMPLIANCE NOTE

Highest-risk item on this map — do not launch without insurance-licensing legal review in every state the placement runs. Many states treat any compensation 'in connection with' insurance solicitation or placement as producer compensation requiring licensure regardless of whether it's structured as a flat sponsorship, per-lead, or per-sale fee, so 'flat fee' framing alone is not automatically safe; the site's own README also bars anything contingent on a sale outright. Keep the placement visually and textually distinct from the calculator's own output so it can never read as the site's recommendation or as a government referral, and ensure the sponsor's copy never states a bare eligibility determination on subsidydropoff's page.

Opt-in newsletter for guide/update alerts — narrow, explicit exception to zero-PII LONGER-TERM

A single voluntary email signup (via a privacy-respecting ESP under a signed DPA) for 'new guide and cliff-update alerts only,' fully decoupled from calculator usage — no calculator inputs, session data, or usage history ever attach to the subscriber record. Signup form lives only on guide pages, never the calculator page.

COMPLIANCE NOTE

This is the one place the README's zero-PII stance needs an explicit, narrow carve-out: state plainly in the privacy policy that an email address is the only PII the site ever collects, is opt-in, is never linked to any calculator session, is not sold or used for ad targeting, and is deletable on request; the calculator itself stays completely untouched — no 'enter your email to see your result' gating, ever — so the core tool's zero-PII claim remains categorically true.

Sponsored/branded guide articles from adjacent non-insurance fintech brands LONGER-TERM

Accept flat-fee sponsored articles from HSA administrators, budgeting apps, or tax-software brands (never insurance carriers or brokers) — e.g., 'Sponsored: How [HSA Provider] Can Lower Your MAGI Below the Cliff' — written or fact-checked in-house.

COMPLIANCE NOTE

Sponsors are non-insurance financial brands, so the broker-licensure constraint doesn't apply, but content must still avoid ever stating a bare eligibility determination on the reader's behalf, and must carry unambiguous 'Sponsored' labeling per FTC native-advertising rules, distinct from the site's editorial voice, so it can't be mistaken for HealthCare.gov guidance or the calculator's own output.

B2B licensing of the calculation engine (embeddable widget/API) to sibling portfolio sites or advisor platforms LONGER-TERM

Package the subsidy-cliff calculation logic as a small licensable widget/API — for internal reuse on a portfolio 'health tools' sibling, or external B2B licensing to fee-only advisor platforms wanting an embeddable cliff calculator — billed as a flat B2B software fee rather than consumer-facing monetization.

COMPLIANCE NOTE

B2B software-licensing revenue is unrelated to insurance sales and doesn't touch the broker-licensing constraint; the API must stay purely computational and stateless, never transmitting or logging licensee end-user data back to subsidydropoff, so it can't quietly become a PII pipeline; every licensee must contractually agree not to present the tool as a government service or bundle it with a bare 'you qualify' determination.

30-DAY PLAN

Remediation & Content Plan

Sequenced so the operational emergency closes first, discoverability scaffolding ships in parallel at near-zero cost, and new content leans on subject-matter writing that already exists in this repository rather than starting from a blank page.

DAYS 1-7

Stop the bleeding: fix the deploy pipeline, fix the data pipeline, ship every zero-dependency SEO/PWA asset in parallel

- P0 — Create the Cloudflare API token (Account > Cloudflare Pages > Edit) and find the Account ID; add CLOUDFLARE_API_TOKEN and CLOUDFLARE_ACCOUNT_ID as GitHub repo secrets (DEPLOY.md steps 1-2) — this single gap is the entire reason nothing has shipped in 22 days
- P0 — Manually run the 'Deploy to Cloudflare Pages' workflow (workflow_dispatch) and confirm verify -> ETL -> synthetic-data guard -> deploy all go green end-to-end, not just typecheck/test
- P0 — Resolve a second, undocumented deploy problem found in the repo: wrangler.toml deploys the bare public/index.html static prototype, but the real, more complete site (hero, live demo, /plan, /methodology — everything in app/ and components/) is a Next.js app that nothing in CI builds or copies into public/. Decide now: wire `next build` output into the Pages deploy (static export or @cloudflare/next-on-pages) so the actual app ships; if that can't land in days 1-3, patch public/index.html as a stopgap so day-one traffic isn't worse than it has to be
- P0 — Attach subsidydropoff.com and www custom domains once the first deploy succeeds (DEPLOY.md step 4)
- P0 — Every URL in src/etl/sources.ts is marked urlVerified:false and has never been hit from a networked environment; verify each CMS PUF download URL actually resolves for plan year 2026 (this is almost certainly the '404' — fix or update the paths before trusting the ETL)
- P0 — Run `npm run etl -- --plan-year=2026 --out=public/data` for real; confirm the CI 'guard against shipping synthetic premiums' step passes because real shards exist, not because it's untested
- P0 — Spot-check ~10 PUF-derived benchmarks against HealthCare.gov window shopping (target ±$2/month), per DEPLOY.md's own pre-launch checklist, before any content leans on these numbers
- P0 — Post-deploy verification: curl /api/health and /api/estimate return live non-synthetic data; confirm CSP/HSTS security headers land
- Ship robots.txt (allow all, reference sitemap.xml) — currently absent entirely
- Ship sitemap.xml seeded with /, /plan, /methodology — currently absent entirely; regenerate as new guide pages ship in weeks 2-3
- Ship manifest.json (name 'Subsidy Dropoff', theme_color from the existing teal/ink tokens, display:standalone) with real 192x192/512x512 PNG icons exported from the existing leaf mark — today there's only an inline SVG data-URI favicon, no manifest at all
- Add JSON-LD (WebApplication/SoftwareApplication) to the homepage — zero structured data exists anywhere on the site today
- Fix og:image: public/og.png exists but is 1024x1024 (square) and isn't even wired into a <meta property="og:image"> tag on the live static prototype; regenerate at 1200x630 and add the tag with width/height on whichever front end ships
- Add the missing Twitter Card tags (twitter:card, title, description, image) to public/index.html — confirmed absent there; the unshipped Next app's layout.tsx already has correct ones, one more reason to prioritize shipping it
- Once live, validate social cards with the Twitter Card Validator, Facebook Sharing Debugger, and LinkedIn Post Inspector

DAYS 8-14

Ship the flagship authority page — go head-to-head with acacalc.com's cliff guide using content the site already wrote but never published

- Write and publish /guide/cliff-vs-phase-out, adapting README's 'Two ways the credit ends' section: natural phase-out (young filers in cheap rating areas who lose the credit gradually well below 400% FPL, for whom managing income down buys nothing) vs. the hard cliff (older households / expensive areas where a large credit survives right up to 400% FPL and one dollar destroys it)
- Lead the guide with the worked example already sitting unused in README/methodology copy: a 60/58 couple at $80,000 in Houston — $4,600 of headroom, $17,501.88/year lost by crossing it
- Match acacalc.com's exact specificity (400.00% still gets a credit, 400.01% gets $0) — this is the precise mechanics query competitors are ranking on
- Add hyperlinks to primary sources on the /methodology Sources list — currently plain-text titles with no href (Rev. Proc. 2026-26, Rev. Proc. 2025-25, HHS poverty guidelines, CMS PUFs are named but not linked); link the actual Rev. Proc. 2026-26 PDF the way README already does internally
- Write one explicit trust/privacy page or section stating plainly: zero PII collected, zero ads, zero affiliate links, zero signup, engine runs client-side — a strictly stronger claim than povertylevelcalculator.com's 'no signup' (which still discloses ad/affiliate monetization) and distinct from QuantCalc's client-side claim (which funnels to a paid tier). One page, not a series — this is positioning, not a blog
- Cross-link homepage -> new guide -> /plan -> /methodology so the page isn't an orphan and the topical cluster reads as one silo
- Add the new guide URL to sitemap.xml

DAYS 15-21

Turn the engine's own unused data into three more guide pages — the genuine informational edge over competitors writing generic advice

- Write /guide/lower-your-magi (HSA, traditional IRA/401(k), and other legal MAGI-reduction levers to stay under 400% FPL) — the direct answer to acasubsidyguide.com's '5 Legal Ways to Lower Your Income' and taxcliffs.co's income-cliff content; frame every tactic against the site's own 'headroom' concept so it pulls readers back to the calculator instead of reading as generic personal-finance content
- Keep the MAGI guide's disclaimer posture identical to the rest of the site — 'estimates only,' 'consult a tax professional' — do not drift into individualized tax advice or scope-creep into a general finance blog
- Write /guide/rev-proc-2026-26 (or a deep-link section of /methodology): plain-English explainer of why an indexed 2027 applicable-percentage table with no bracket above 400% FPL is the dispositive signal the cliff is real for plan year 2027 — content no competitor in the survey appears to have this specifically; link the primary IRS PDF directly
- Write /guide/state-marketplace-handoff surfacing the 51-jurisdiction FFM/SBE-FP/SBE dataset already built in src/data/exchanges.ts: which states run their own exchange, and which of those layer a state subsidy on top of the federal credit (where a federal-only number would understate what a resident actually gets)
- Treat the state guide as one well-built aggregate page, not 50 thin programmatic pages like acasubsidyguide.com/taxcliffs.co — write down full state-by-state pages as post-30-day backlog rather than building them now
- Add FAQPage JSON-LD to whichever new guide reads naturally as Q&A (cliff-vs-phase-out or the MAGI guide)
- Update sitemap.xml with all three new URLs; internal-link the three guides to each other and from the homepage/nav so the cluster is legible to crawlers

DAYS 22-30

Get found, get verified, get positioned — indexing, closing the trust/verification debt, tightening, not more sprawling content

- Submit sitemap.xml to Google Search Console and Bing Webmaster Tools and verify domain ownership — a direct search for subsidydropoff.com currently returns nothing at all, confirming the site is fully unindexed
- Request indexing for the homepage and each of the four new guide pages individually in Search Console
- Close out README's open 'Verification status' items now that published content depends on them: re-verify Rev. Proc. 2026-26 brackets against the primary PDF (currently secondary-concordant, never actually read), populate Alaska/Hawaii poverty guidelines (engine currently refuses both states), resolve or caveat the contested PY2027 open-enrollment end date (vacated in City of Columbus v. Kennedy, on appeal)
- Publish one short 'How to evaluate any ACA subsidy calculator' checklist page (data provenance, PII handling, ad/affiliate disclosure) — genuinely useful content that functions as implicit competitive positioning without naming or disparaging specific competitors
- QA pass: confirm every new guide renders correct social cards in the Twitter/LinkedIn/Facebook debuggers, confirm no page is an orphan (nav + footer + sitemap all agree), run Lighthouse and confirm the new manifest.json actually improves the PWA score
- Re-confirm the ETL/CI pipeline is still green a full week after first light — the weekly Monday cron in deploy.yml should have fired at least once by now, the first real proof the 'no one has to remember to do this' claim in DEPLOY.md holds
- Write down (do not build) the beyond-day-30 backlog: full 50-state programmatic pages, a CSR/cost-sharing explainer, an OBBB repayment-cap-repeal deep dive matching TaxPayers.net's 2026 nuance — keeps next month's scope explicit instead of creeping into this one

BEYOND DAY 30

Full 50-state programmatic pages, a CSR/cost-sharing explainer, and an OBBB repayment-cap-repeal deep dive matching TaxPayers.net's 2026 nuance are explicitly out of scope for this plan — written down as backlog, not built, so next month's scope stays a deliberate decision rather than creep.